

Let's learn about **satellites!**

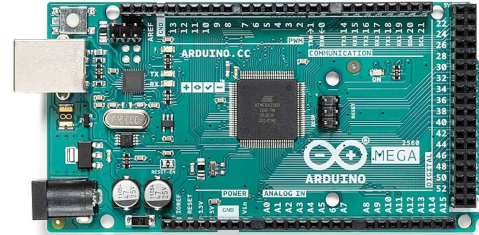
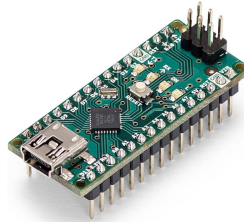
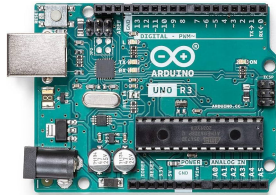
Space mission and **satellite** design course

Bonus Lesson

Introduction to Arduino Programming

Arduino is an open-source platform:

- Hardware: Arduino UNO, Arduino Nano, Mega...



- Software: Arduino IDE



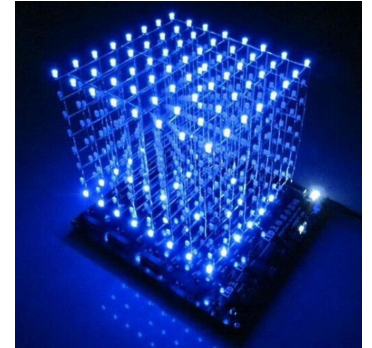
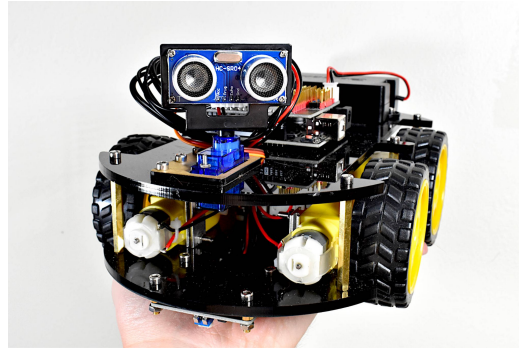
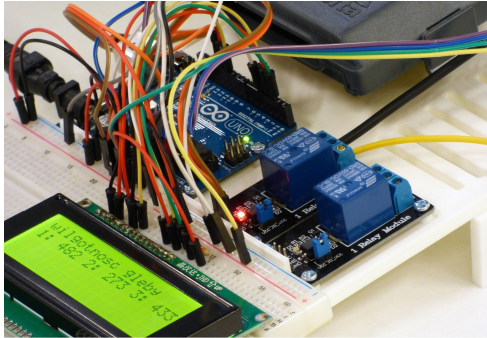
Introduction to Arduino Programming

- Accessible Entry Point: Cheap, user-friendly interface, no need for a deep technical background.
- Learning Tool: Arduino provides a hands-on experience that helps learners understand principles of electronics and programming..
- Rapid Prototyping: It allows for quick and efficient testing of ideas and concepts,
- Versatility in Projects: Arduino's versatility allows it to be employed in a broad range of projects.
- Open-Source Collaboration: Large collaborative web community. Users worldwide contribute to a vast repository of libraries and projects, promoting knowledge-sharing and collective problem-solving.



Introduction to Arduino Programming

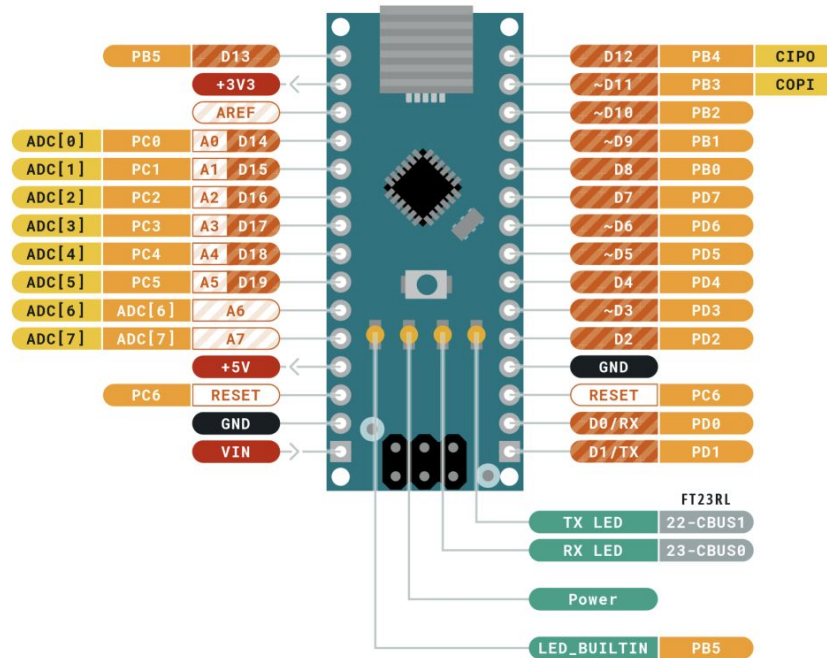
Arduino is extremely versatile:



Arduino: Electrical Overview

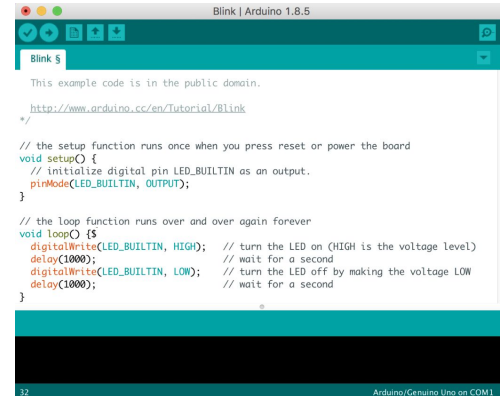
- Microcontroller: The microcontroller is a small integrated circuit (IC) that contains a processor (CPU), memory, and input/output peripherals. The most commonly used microcontroller on Arduino boards is the Atmel AVR series.
- Input/Output Pins: I/O pins are the connectors on the Arduino board that allow it to interact with the external world. These pins can be configured as either inputs or outputs. Input pins are used to receive signals, such as data from sensors, while output pins are used to send signals, such as controlling LEDs or motors.
- Power Supply: The power supply component refers to how the Arduino board is powered. Arduino boards can be powered through various means, including USB, External Power Supply and Battery.

Arduino Nano



Introduction to Arduino Programming

Arduino can be programmed via a USB cable through the Arduino IDE, allowing for seamless and convenient uploading of code to the Arduino board in a matter of seconds.



Your First Arduino Program

```
// Hello World program - Blinking LED

// Pin connected to the LED
const int ledPin = 13;

void setup() {
  // initialize digital pin as an output.
  pinMode(ledPin, OUTPUT);
}

void loop() {
  // turn the LED on (HIGH) then off (LOW) in sequence
  digitalWrite(ledPin, HIGH);
  delay(1000); // wait for a second
  digitalWrite(ledPin, LOW);
  delay(1000); // wait for a second
}
```

Blinking an LED

The program begins by defining a constant variable (ledPin) to represent the pin connected to an LED on the Arduino board.

Arduino programs have a basic structure consisting of two essential functions: **setup()** and **loop()**.

Your First Arduino Program

```
// Hello World program - Blinking LED

// Pin connected to the LED
const int ledPin = 13;

void setup() {
  // initialize digital pin as an output.
  pinMode(ledPin, OUTPUT);
}

void loop() {
  // turn the LED on (HIGH) then off (LOW) in sequence
  digitalWrite(ledPin, HIGH);
  delay(1000); // wait for a second
  digitalWrite(ledPin, LOW);
  delay(1000); // wait for a second
}
```

Blinking an LED

The **setup()** function is called once when the Arduino board starts running the program.

It is primarily used for initializing variables, configuring pins, and performing setup tasks. In the "Hello World" program, the `pinMode()` function is used in `setup()` to define `ledPin` as an output, indicating it will be used to send signals to the LED.

Your First Arduino Program

```
// Hello World program - Blinking LED

// Pin connected to the LED
const int ledPin = 13;

void setup() {
  // initialize digital pin as an output.
  pinMode(ledPin, OUTPUT);
}

void loop() {
  // turn the LED on (HIGH) then off (LOW) in sequence
  digitalWrite(ledPin, HIGH);
  delay(1000); // wait for a second
  digitalWrite(ledPin, LOW);
  delay(1000); // wait for a second
}
```

Blinking an LED

The **loop()** function is the heart of the Arduino program.

It runs continuously after the setup() function completes. Code placed in loop() executes in a loop, allowing for repeated actions.

In this program, the digitalWrite() function within loop() turns the LED on (HIGH) and off (LOW) in sequence, creating a blinking effect.

Variables and Data Types

```
// Variables and Data Types

int integerVar = 42;    // Integer variable
float floatVar = 3.14; // Floating-point variable
char charVar = 'A';    // Character variable

void setup() {
    // Your setup code here
}

void loop() {
    // Your main code here
}
```

A variable is a named storage location that holds a value. Variables allow you to store and manipulate data in your program.

Data types define the type of data that a variable can hold, such as integers, floating-point numbers, or characters.

Understanding and correctly using variables and data types are fundamental concepts in Arduino programming.

Common Data Types in Arduino:

- int (Integer): Purpose: Used for storing whole numbers (integers) without decimal points.
- float (Floating-Point): Purpose: Used for storing numbers with decimal points.
- char

Variables and Data Types

```
// Variables and Data Types

int integerVar = 42;    // Integer variable
float floatVar = 3.14; // Floating-point variable
char charVar = 'A';    // Character variable

void setup() {
    // Your setup code here
}

void loop() {
    // Your main code here
}
```

Common Data Types in Arduino:

- int (Integer): Used for storing whole numbers (integers) without decimal points.
- float (Floating-Point): Used for storing numbers with decimal points.
- char (Character): Used for storing a single character (letter, digit, or symbol).

Variable Declarations and Assignments:

Variable Declaration: This is the process of specifying the data type and name of a variable before it is used in the program. It informs the compiler about the type of data the variable will store. Variable Assignment:

Variables and Data Types

```
// Variables and Data Types

int integerVar = 42;    // Integer variable
float floatVar = 3.14;  // Floating-point variable
char charVar = 'A';    // Character variable

void setup() {
    // Your setup code here
}

void loop() {
    // Your main code here
}
```

Variable Declaration: This is the process of specifying the data type and name of a variable before it is used in the program. It informs the compiler about the type of data the variable will store.

Variable Assignment: This is the process of giving a variable a specific value. The = operator is used for assignment.

Combined Declaration and Assignment: It is common to declare and assign a variable in a single line.

Control Structures

Control structures are fundamental elements in programming that allow you to control the flow of your code.

In Arduino programming, two primary control structures are widely used: if statements and loops (for and while).

These structures enable you to make decisions based on certain conditions and repeat specific actions, respectively.

Control Structures

If Statements

Purpose: if statements are used for decision-making in your code.

They allow you to execute a block of code only if a specified condition is true.

```
int temperature = 25;

if (temperature > 20) {
    // Code to execute if temperature is greater than 20
}
```


Control Structures

Loops allow you to repeat a block of code multiple times, either for a predefined number of iterations (for loop) or until a certain condition is met (while loop).

```
const int ledPin = 13;

void setup() {
  pinMode(ledPin, OUTPUT);
}

void loop() {
  for (int i = 0; i < 5; i++) {
    digitalWrite(ledPin, HIGH); // Turn the LED on
    delay(500); // Wait for 500 milliseconds (0.5 seconds)
    digitalWrite(ledPin, LOW); // Turn the LED off
    delay(500); // Wait for 500 milliseconds (0.5 seconds)
  }
  delay(2000); // Pause for 2 seconds before restarting the loop
}
```

In this example, a for loop is used to blink an LED connected to pin 13 five times.

Inside the loop, the LED connected to pin 13 is turned on and off with delays to create a blinking effect.

After the for loop completes, there's a pause of 2 seconds before the loop restarts.

Control Structures

Loops allow you to repeat a block of code multiple times, either for a predefined number of iterations (for loop) or until a certain condition is met (while loop).

```
const int buttonPin = 2;
const int ledPin = 13;

void setup() {
  pinMode(buttonPin, INPUT);
  pinMode(ledPin, OUTPUT);
}

void loop() {
  while (digitalRead(buttonPin) == HIGH) {
    digitalWrite(ledPin, HIGH); // Turn the LED on while the button is pressed
  }
  digitalWrite(ledPin, LOW); // Turn the LED off when the button is released
}
```

In this example, a while loop is used to continuously check if a button connected to pin 2 is pressed. When pressed, the built-in LED on pin 13 is turned on.

While the button is pressed, the LED on pin 13 remains on. When the button is released, the LED is turned off.

Functions & Code Modularity

A function is a self-contained block of code that performs a specific task.

Functions are used to break down a program into smaller, manageable pieces, making the code more organized, readable, and reusable.

The main roles of functions in programming include promoting modularity, code reuse, and enhancing readability.

Functions & Code Modularity

In Arduino programming, creating and using functions follows a straightforward process.

Function Declaration: Declare a function by specifying its return type, name, and parameters (if any). For example:

```
void myFunction(int parameter1, float parameter2) {  
    // Function code goes here  
}
```

Function Definition: Define the function by providing the actual code that executes when the function is called.

```
void myFunction(int parameter1, float parameter2) {  
    // Code to be executed  
    // ...  
}
```

Functions & Code Modularity

Function Call: Call the function from the setup() or loop() function, or from another function.

```
void setup() {  
  // Other setup code  
  myFunction(10, 3.14); // Calling the function with arguments  
}
```

Functions & Code Modularity

Importance of Modular Code:

- **Modularity:** Breaking down a program into functions creates modular code, where each function addresses a specific aspect of the program's functionality. This modularity makes the code easier to understand and maintain.
- **Code Reusability:** Functions allow you to reuse code in different parts of your program or in other projects. Once a function is defined, you can call it whenever you need that specific functionality.
- **Readability:** Modular code enhances readability by providing a clear structure. Functions act as building blocks, and the main program or other functions become more readable and concise.

Functions & Code Modularity

Importance of Modular Code:

- **Ease of Maintenance:** When a program is modular, it is easier to locate and fix issues or make updates. Changes to one function do not necessarily affect others, making maintenance more straightforward.
- **Collaboration:** In collaborative programming efforts, modular code facilitates teamwork. Different team members can work on separate functions, and integration becomes more manageable.

In Arduino programming, where projects can vary in complexity, using functions becomes especially valuable. Whether you're building a simple LED blinking project or a complex IoT application, functions contribute to code organization and efficiency.

Analog Signals vs Digital Signals - Input & Output

Analog Signals: Analog signals are continuous and can have an infinite number of values within a given range. In the physical world, examples include varying voltage levels, sound waves, and temperature.

Arduino uses analog signals to represent values between 0 and 1023 using a 10-bit analog-to-digital converter (ADC).

Digital Signals: Digital signals are discrete and can only have two possible values: HIGH or LOW, 1 or 0.

Digital signals are commonly used for binary communication, where information is represented using a series of bits.

analogRead() and analogWrite() Functions:

`analogRead()` and `analogWrite()` are built-in functions in the Arduino programming environment, facilitating the reading of analog signals from sensors and the generation of signals for controlling analog devices on Arduino boards.

`analogRead()` Function: Used to read analog voltage values from analog pins on the Arduino. The returned value is a digital representation of the analog voltage, ranging from 0 to 1023.

```
int sensorValue = analogRead(A0); // Read analog input from pin A0
```

`analogWrite()` Function: Used to generate a PWM (Pulse Width Modulation) signal, allowing you to simulate an analog output. It is often used to control the brightness of an LED or the position of a servo motor.

```
analogWrite(9, 128); // Write PWM value to pin 9 (range: 0 to 255)
```

analogRead() and analogWrite() Functions:

Examples using analog sensors and actuators:

Analog Sensor Example (Potentiometer): Connect a potentiometer to an analog pin, and read its value using analogRead()

```
int potPin = A0;  
int potValue = analogRead(potPin);
```

analogRead() and analogWrite() Functions:

Analog Actuator Example (Servo Motor): Connect a servo motor to a PWM-enabled pin, and control its position using analogWrite()

```
#include <Servo.h>

Servo myServo;

void setup() {
  myServo.attach(9); // Attach the servo to pin 9
}

void loop() {
  int angle = analogRead(A0); // Read analog input
  angle = map(angle, 0, 1023, 0, 180); // Map the value to servo angle
  myServo.write(angle); // Set the servo position
}
```

digitalRead() and digitalWrite() Functions:

digitalRead() and digitalWrite() are built-in functions in the Arduino programming environment, facilitating the reading of digital signals from sensors and the generation of signals for controlling digital devices on Arduino boards.

digitalRead() Function: digitalRead() reads the digital state (HIGH or LOW) of a specified digital pin.

digitalWrite() Function: digitalWrite() sets the digital state (HIGH or LOW) of a specified digital pin.

```
int buttonState = digitalRead(2); // Reads the state of a button connected to pin 2
digitalWrite(13, HIGH); // Sets pin 13 to a HIGH state, turning on an LED
```

Analog Signals vs Digital Signals - Input & Output

It is crucial to know whether we are dealing with analog or digital signals in Arduino I/O because it determines how we interpret and interact with the data.

Analog signals provide a continuous range of values, often representing real-world measurements like sensor readings, where precise values matter.

Digital signals, on the other hand, are binary, representing only two states (HIGH or LOW), suitable for tasks like controlling LEDs or reading the state of a button.

Understanding the nature of the signal ensures proper handling and processing, allowing for accurate communication between the Arduino and external devices.

...Write(), ...Read() & pinMode() Correlation

In Arduino programming, the `digitalWrite()` function is specifically designed for declared output pins, and the `digitalRead()` function is used for declared input pins.

This means that you can use `write` to set the output state (HIGH or LOW) of pins that have been configured as outputs using `pinMode()`. Conversely, you can use `read` to obtain the digital state (HIGH or LOW) from pins that have been configured as inputs.

The same thing applies for `analogWrite()` and `analogRead()` in the same logic way.

...Write(), ...Read() & pinMode() Correlation

```
const int ledPin = 13;
const int buttonPin = 2;

void setup() {
  pinMode(ledPin, OUTPUT); // Declare pin 13 as an output
  pinMode(buttonPin, INPUT); // Declare pin 2 as an input
}

void loop() {
  int buttonState = digitalRead(buttonPin); // Read the state of the button

  if (buttonState == HIGH) {
    digitalWrite(ledPin, HIGH); // Turn on the LED if the button is pressed
  } else {
    digitalWrite(ledPin, LOW); // Turn off the LED if the button is not pressed
  }
}
```

...Write(), ...Read() & pinMode() Correlation

```
#include <Servo.h>

const int potPin = A0;
const int servoPin = 9;

void setup() {
    pinMode(potPin, INPUT); // Declare A0 as an input for the potentiometer
    pinMode(servoPin, OUTPUT); // Declare pin 9 as an output for the servo motor
}

void loop() {
    int potValue = analogRead(potPin); // Read potentiometer value
    int servoAngle = map(potValue, 0, 1023, 0, 180); // Map to servo angle range

    // Perform additional actions based on potValue...

    Servo myServo; // Create a servo object
    myServo.attach(servoPin); // Attach the servo to pin 9
    myServo.write(servoAngle); // Set the servo position
    delay(15); // Add a short delay for servo movement
}
```


Working with Libraries

Libraries are pre-written sets of functions and code that simplify complex tasks, enabling users to focus on higher-level aspects of their projects.

Libraries encapsulate functionality, making it easy to integrate additional features without the need to write extensive code from scratch. They contribute to code modularity, reusability, and efficiency.

Arduino libraries can be found and accessed through the built-in Library Manager in the Arduino IDE, the official Arduino website, GitHub, third-party websites, and community forums, providing a rich resource of pre-written code and functionalities for use in Arduino projects.



Working with Libraries

Libraries have to be included at the beginning of your code, generally before the `setup()` function, to ensure that the necessary functions and definitions from the libraries are available throughout the entire Arduino sketch, enabling seamless integration and utilization of their functionalities.

Common libraries:

- Servo Library: facilitates the control of servo motors. `#include <Servo.h>`
- Wire Library: is used for I2C communication, enabling Arduino boards to communicate with other devices in a network. `#include <Wire.h>`
- LiquidCrystal: Provides functions for interfacing with character LCD displays.
- Adafruit NeoPixel: Enables control of NeoPixel RGB LEDs, allowing for dynamic lighting effects.

Working with Libraries

Examples of using libraries in projects:

```
#include <Servo.h>

Servo myServo; // Create a servo object

void setup() {
  myServo.attach(9); // Attach the servo to pin 9
}

void loop() {
  myServo.write(90); // Set the servo position to 90 degrees
  delay(1000); // Wait for 1 second
  myServo.write(180); // Set the servo position to 180 degrees
  delay(1000); // Wait for 1 second
}
```

Working with Libraries

Examples of using libraries in projects:

```
#include <Wire.h>

void setup() {
    Wire.begin(); // Initialize the I2C communication
}

void loop() {
    Wire.beginTransmission(8); // Start communication with device at address 8
    Wire.write("Hello"); // Send data
    Wire.endTransmission(); // End transmission
    delay(1000); // Wait for 1 second
}
```

Serial Communication

Serial communication is a fundamental aspect of Arduino programming, enabling bidirectional data transfer between the Arduino board and external devices.

The `Serial.begin()` function initializes the communication at a specified baud rate, while `Serial.print()` and `Serial.println()` allow for the transmission of data to the serial port for debugging and monitoring purposes.

On the receiving end, `Serial.read()` is utilized to read incoming data, often employed for processing commands or information from connected devices.

This capability enhances the versatility of Arduino projects, facilitating real-time monitoring, debugging, and interactive data exchange with external components or systems.

```
void setup() {  
    Serial.begin(9600); // Initialize serial communication at 9600 baud  
    pinMode(LED_BUILTIN, OUTPUT); // Set built-in LED as an output  
}  
  
void loop() {  
    // Sending data: Sensor value for monitoring  
    int sensorValue = analogRead(A0);  
    Serial.print("Sensor Value: ");  
    Serial.println(sensorValue);  
  
    // Receiving data: Process commands from external source  
    if (Serial.available() > 0) {  
        char command = Serial.read();  
        switch (command) {  
            case 'A':  
                // Turn on the LED for action A  
                digitalWrite(LED_BUILTIN, HIGH);  
                break;  
            case 'B':  
                // Turn off the LED for action B  
                digitalWrite(LED_BUILTIN, LOW);  
                break;  
            // Add more cases as needed  
        }  
    }  
}
```

This Arduino code initializes serial communication, sends sensor values for monitoring, and listens for incoming commands, allowing external sources to control actions, such as turning an LED on or off based on received commands.

Serial communication can be visualized in the Arduino IDE's Serial Monitor, where real-time data, such as sensor readings, is displayed.

Additionally, the Serial Monitor serves as a platform for sending commands directly to the Arduino board, enabling interactive control of connected devices or the execution of specific actions within the code.

Arduino Basic Functions Recap

setup():

- Executes once at the beginning of the program.
- Used for initialization tasks such as defining pin modes and setting up communication.

loop():

- Executes repeatedly after the setup() function.
- Contains the main code logic and runs in a loop as long as the Arduino is powered.

pinMode(pin, mode):

- Configures a specified pin as INPUT or OUTPUT.
- Important for defining how a pin should be used.

```
pinMode(13, OUTPUT); // Example: Set pin 13 as an output
```

Arduino Basic Functions Recap

digitalWrite(pin, value): `digitalWrite(13, HIGH);`

- Writes a HIGH or LOW value to a specified digital pin.
- Used for turning LEDs on/off, controlling relays, etc.

digitalRead(pin) Function: `int buttonState = digitalRead(2);`

- Reads the digital value (HIGH or LOW) from a specified digital pin.
- Commonly used for reading button states or digital sensors.

analogWrite(pin, value): `analogWrite(9, 128);`

- Writes an analog value (PWM) to a specified pin.
- Useful for controlling the brightness of an LED or the speed of a motor.

analogRead(pin): `int sensorValue = analogRead(A0);`

- Reads the analog value (0 to 1023) from a specified analog pin.
- Used for reading values from sensors like potentiometers.

Arduino Basic Functions Recap

Serial.begin(baudRate): `Serial.begin(9600);`

- Initializes serial communication at a specified baud rate.
- Essential for using the Serial Monitor for debugging and communication.

Serial.print(data) and Serial.println(data): `Serial.print("Hello, World!");`

- Print data to the Serial Monitor for debugging and monitoring.

delay(milliseconds): `delay(1000);`

- Pauses the program for a specified number of milliseconds.
- Used for creating delays in the program's execution.

Connecting Sensors and Actuators to Arduino



Actuators



Introduction to Sensors and Actuators

Arduino cannot autonomously acquire data or engage with the environment.

To expand its capabilities, you must integrate sensors and actuators with your Arduino board. This connection opens up a realm of possibilities for data acquisition and interaction, enabling a diverse range of applications.

A **sensor** is a device that detects and measures physical properties or environmental conditions, converting them into electrical signals for processing. In contrast, an **actuator** is a component that receives signals from a controller or system and translates them into physical actions, effecting changes in the environment or a connected system.

Sensors gather data, while actuators enable the execution of actions based on that data.

Sensors and Actuators: Examples

Sensors:

Actuators:

Sensors and Actuators: Examples

Sensors:

- Photoresistor
- Touch sensor
- Temperature sensor (thermometer)
- Humidity sensor (hygrometer)
- Gyroscope
- Accelerometer
- Water flow sensor
- Pressure sensor
- Ultrasonic sensor
- Color sensor
- Proximity sensor
- etc.

Actuators:

- DC Motors
- Servo motors
- Linear actuators
- Piezoelectric actuators
- Hydraulic actuators
- Pneumatic actuators
- Solenoids
- etc.

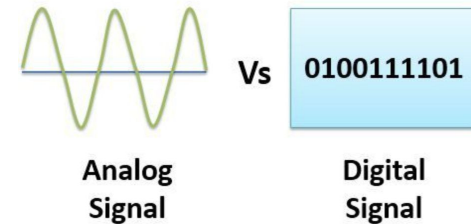
Understanding Sensors and Actuators: Analog vs Digital

In the realm of electronics, devices can be broadly categorized into analog sensors, digital sensors, and actuators based on their functionality.

Analog sensors provide a continuous range of values, often in response to physical phenomena like light or temperature.

Digital sensors, on the other hand, produce discrete, binary outputs, typically representing states like "on" or "off."

Actuators, meanwhile, are devices capable of physical movement or action in response to electronic signals.



Understanding Sensors and Actuators: Pins

Understanding the pin configuration is crucial when connecting these devices to an Arduino or any microcontroller.

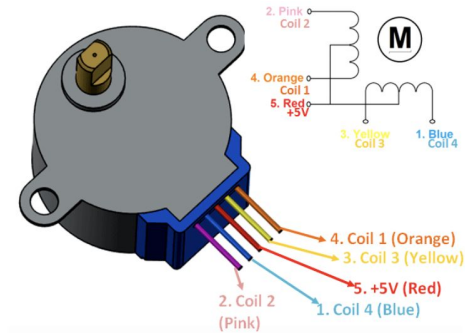
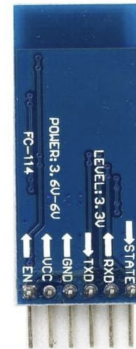
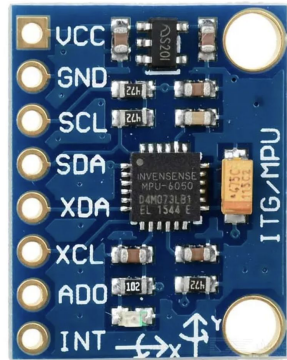
For both sensor and actuator connections:

- **Power (VCC)**: This crucial pin supplies operating voltage. Ensuring compatibility with device specifications is essential for proper functionality.
- **Ground (GND)**: Serving as a common electrical reference, the ground pin establishes connectivity, linking the device's ground to the microcontroller's ground.
- **Signal Pins (for Sensors) / Control Pins (for Actuators)**:
 - Sensors: Analog sensors transmit varying voltage levels through a signal pin, representing the sensed quantity.
 - Digital sensors use a digital output pin for binary information.
 - Actuators: Control pins receive electronic signals from the microcontroller, dictating the desired movement or action.

Powering Sensors and Actuators

When connecting devices to an Arduino, it's crucial to align their voltage requirements with the microcontroller's capabilities to ensure optimal performance and prevent potential damage.

This involves connecting the device's VCC (power) pin to the Arduino's power supply, choosing either the 5V or 3.3V pin based on the device's specified voltage needs. External power supplies can be used too.



Grounding Sensors and Actuators

Connecting the device's GND (ground) pin to the Arduino's GND pin is essential for creating a shared ground reference in your electronic setup.

This common ground ensures that both the device and the Arduino operate within the same electrical framework, facilitating smooth communication.

A stable ground connection is vital for accurate sensor readings and proper actuator operation, especially when using an external power supply. It minimizes electrical noise and interference, contributing to the reliability and precision of your electronic projects interfaced with the Arduino.



Signal
ground



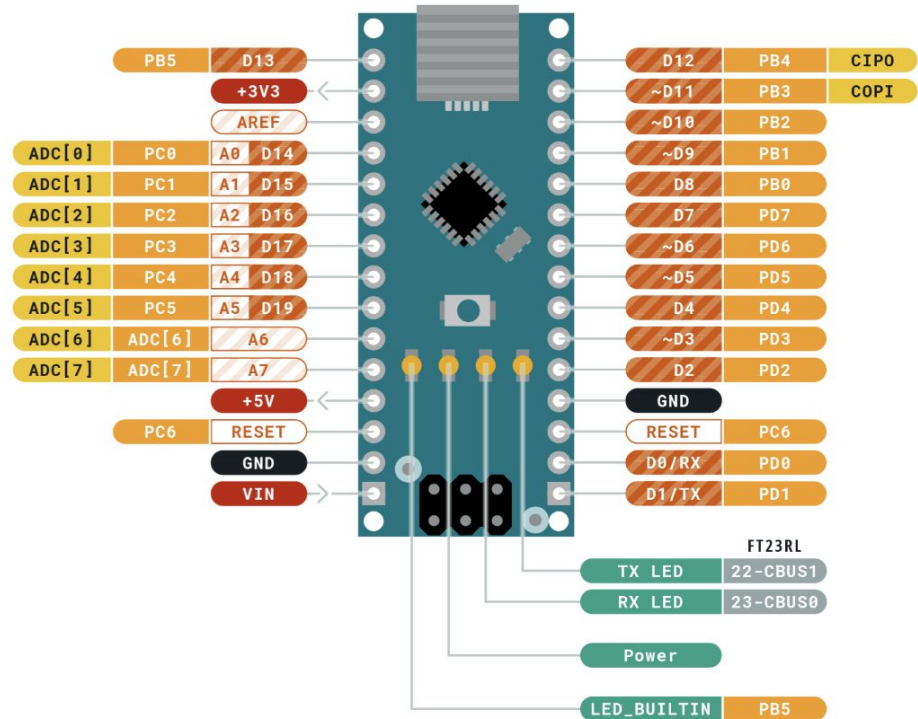
Chassis
ground



Earth
ground

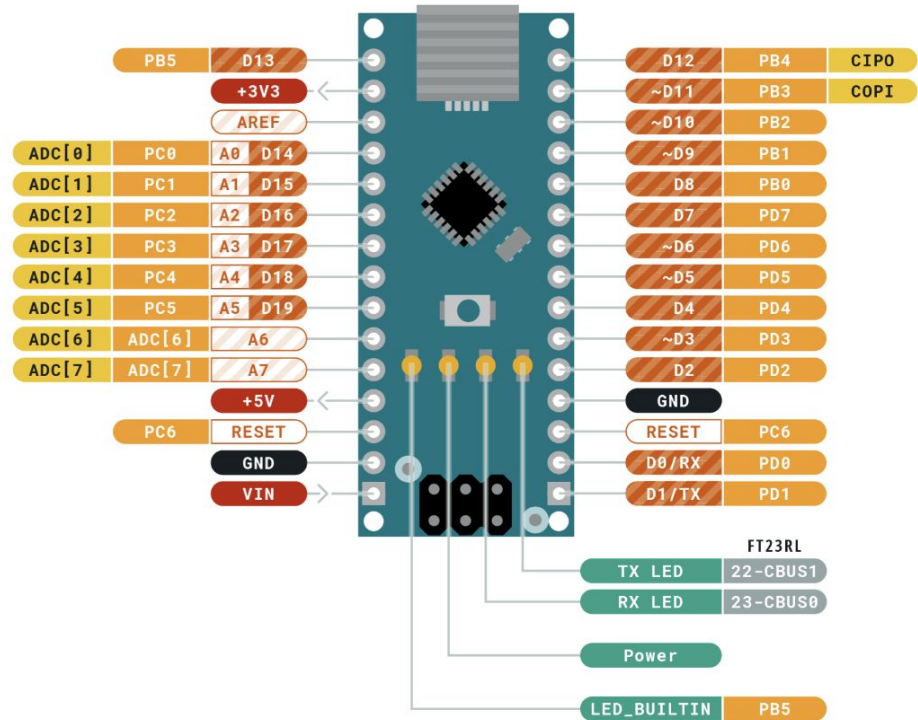
Signal Connections for Sensors and Actuators

- Analog Sensor: Connect the sensor's signal wire to one of the Arduino Nano's analog pins (A0-A5).
- Digital Sensor: Connect the sensor's digital signal wire to one of the Arduino Nano's digital pins (2-13).



Signal Connections for Sensors and Actuators

- On an Arduino Nano, digital pins (2-13) can typically be used to control actuators.
- The following digital pins support Pulse Width Modulation (PWM), allowing you to control the intensity of analog-like signals: D3, D5, D6, D9, D10, D11. These pins are useful for controlling things like the brightness of an LED or the speed of a motor.



Reading Sensor Data in Arduino

To acquire data from sensors in Arduino code, we use specific functions:

- `analogRead()` Function:
 - Usage: Employ `analogRead()` to capture data from analog sensors. Analog sensors provide a continuous range of values (e.g., light or temperature sensors), and `analogRead()` reads the analog voltage on a specified analog pin and converts it to a digital value (0-1023).

```
int sensorPin = A0; // Analog sensor connected to A0
int sensorValue = analogRead(sensorPin); // Read sensor data
```

- `digitalRead()` Function:
 - Usage: Utilize `digitalRead()` for digital sensors, which produce binary outputs (e.g., motion or proximity sensors). It reads the state (HIGH or LOW) of a specified digital pin.

```
int sensorPin = 2; // Digital sensor connected to digital pin 2
int sensorState = digitalRead(sensorPin); // Read sensor state (HIGH or LOW)
```

Controlling Actuators in Arduino

To command actuators in Arduino code, use specific functions:

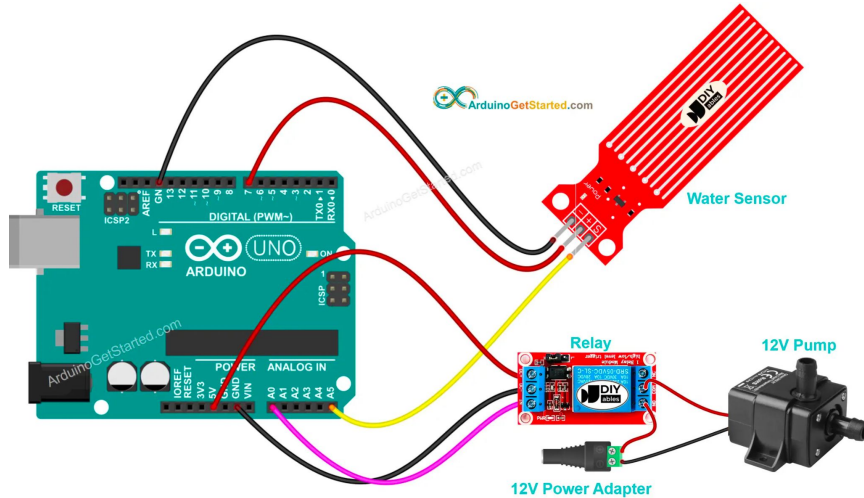
- `digitalWrite()` Function:
 - Usage: Employ `digitalWrite()` for digital actuators, such as turning on/off an LED or controlling a relay. It sets the specified digital pin to either HIGH (5V) or LOW (0V).

```
int actuatorPin = 3; // Digital actuator connected to digital pin 3
digitalWrite(actuatorPin, HIGH); // Turn ON the digital actuator
```

- `analogWrite()` Function:
 - Usage: Utilize `analogWrite()` for analog actuators like servo motors. It produces a PWM signal on the specified pin, enabling varied levels of output.

```
int actuatorPin = 6; // PWM-enabled actuator connected to digital pin 6
int speedValue = 128; // Set the speed value (0-255)
analogWrite(actuatorPin, speedValue); // Control the analog actuator speed
```

Example



```
#define RELAY_PIN    A0 // Pin for pump control
#define POWER_PIN    7  // Pin providing power to the water sensor
#define SIGNAL_PIN   A5 // Pin reading water sensor data
#define THRESHOLD    300 // Water detection threshold

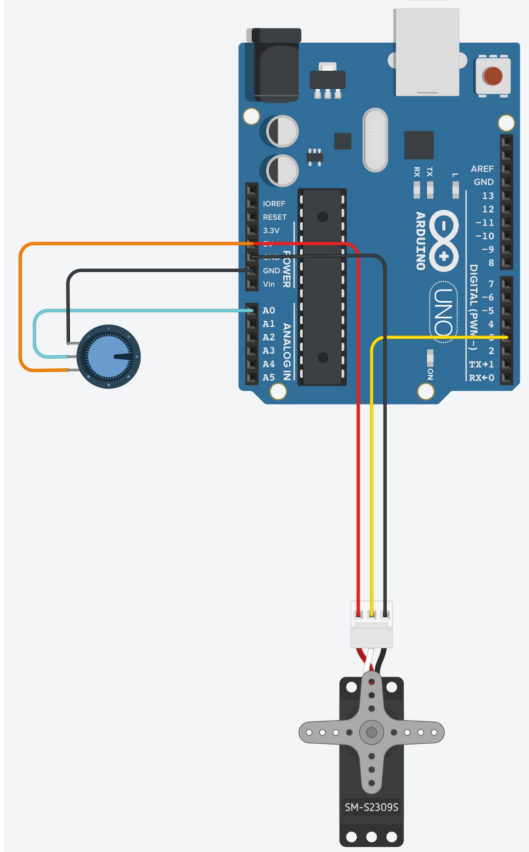
void setup() {
    Serial.begin(9600);
    pinMode(RELAY_PIN, OUTPUT); // Pump control pin setup
    pinMode(POWER_PIN, OUTPUT); // Power pin setup for water sensor
    digitalWrite(POWER_PIN, LOW); // Water sensor OFF
    digitalWrite(RELAY_PIN, LOW); // Pump OFF
}

void loop() {
    digitalWrite(POWER_PIN, HIGH); // Water sensor power ON
    delay(10); // Short delay
    int value = analogRead(SIGNAL_PIN); // Read water sensor data
    digitalWrite(POWER_PIN, LOW); // Water sensor power OFF

    if (value > THRESHOLD) {
        Serial.println("Water detected"); // Print detection message
        digitalWrite(RELAY_PIN, HIGH); // Pump ON
    } else {
        digitalWrite(RELAY_PIN, LOW); // Pump OFF
    }

    delay(1000); // Pause for 1 sec to avoid frequent readings
}
```

Example



```
#include <Servo.h>

#define SERVO_PIN 3    // Pin for the servo motor
#define POT_PIN    A0  // Pin for the potentiometer

Servo myServo;  // Create a Servo object

void setup() {
    myServo.attach(SERVO_PIN); // Attach the servo to its pin
}

void loop() {
    int potValue = analogRead(POT_PIN); // Read the potentiometer value (0-1023)
    // Map potentiometer value to servo angle (0-180)
    int servoAngle = map(potValue, 0, 1023, 0, 180);

    myServo.write(servoAngle); // Set the servo to the calculated angle
    delay(15); // Small delay for smooth servo movement
}
```